

# Projet – Graphes et algorithmique

Boudouma Lambarki Bahir, Lepretre Matys, Fellahi Tarik, Watel Noa

12 octobre 2025

# Table des matières

|          |                                                                                        |           |
|----------|----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                    | <b>3</b>  |
| <b>2</b> | <b>Présentation du problème</b>                                                        | <b>4</b>  |
| 2.1      | NP-complétude et NP-difficulté . . . . .                                               | 4         |
| <b>3</b> | <b>Modélisation mathématique</b>                                                       | <b>5</b>  |
| 3.1      | Définition du graphe . . . . .                                                         | 5         |
| 3.2      | Partition des points . . . . .                                                         | 5         |
| 3.3      | Poids des points de livraison . . . . .                                                | 5         |
| 3.4      | Coefficients de pondération . . . . .                                                  | 6         |
| 3.5      | Variables de décision . . . . .                                                        | 6         |
| 3.6      | Contraintes . . . . .                                                                  | 6         |
| 3.7      | Fonction objectif . . . . .                                                            | 6         |
| 3.7.1    | Relation entre les poids $w_e$ et $w_n$ . . . . .                                      | 7         |
| <b>4</b> | <b>Construction et représentation des graphes</b>                                      | <b>8</b>  |
| 4.1      | Représentation des graphes . . . . .                                                   | 8         |
| <b>5</b> | <b>Résolution du problème</b>                                                          | <b>9</b>  |
| 5.0.1    | Illustration d'une petite instance du problème – Algorithme Brute Force . . . . .      | 9         |
| 5.1      | Algorithmes d'approximation (Christofides) . . . . .                                   | 10        |
| 5.1.1    | Algorithme de tournée pondérée avec backbone (Christofides express + normal) . . . . . | 10        |
| 5.1.2    | Algorithme hybride : backbone de Christofides et insertion gloutonne                   | 12        |
| 5.2      | Comparaison des complexités algorithmiques . . . . .                                   | 14        |
| <b>6</b> | <b>Métaheuristique : principe général et application</b>                               | <b>16</b> |
| 6.1      | Principe général . . . . .                                                             | 16        |
| 6.2      | Application au TSP pondéré (cas du livreur) . . . . .                                  | 16        |
| 6.3      | Implémentation résumée . . . . .                                                       | 17        |
| 6.4      | Résultats expérimentaux (exemple) . . . . .                                            | 17        |
| 6.5      | Complexité et intérêt pratique . . . . .                                               | 18        |
| 6.6      | Conclusion . . . . .                                                                   | 18        |
| <b>7</b> | <b>Indicateur de performance</b>                                                       | <b>19</b> |
| 7.0.1    | Express Priority Index (EPI) . . . . .                                                 | 19        |
| 7.0.2    | Duration Usage Index (DU) . . . . .                                                    | 19        |
| 7.0.3    | Analyse comparative . . . . .                                                          | 20        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>8</b> | <b>Analyse critique et conclusion</b>                     | <b>21</b> |
| 8.0.1    | Limites de la simulation . . . . .                        | 21        |
| 8.0.2    | Optimisation des paramètres $\alpha$ et $\beta$ . . . . . | 21        |
| 8.0.3    | Présentation des trois algorithmes développés . . . . .   | 21        |
| 8.0.4    | Comparaison synthétique . . . . .                         | 23        |
| 8.0.5    | Conclusion . . . . .                                      | 23        |

# Chapitre 1

## Introduction

Dans un contexte où les échanges de biens et de services s'intensifient, la logistique urbaine devient un enjeu crucial pour les entreprises de transport. Optimiser les tournées de livraison permet non seulement de réduire les coûts opérationnels, mais aussi de limiter l'empreinte environnementale et d'améliorer la qualité de service.

Cependant, dans la réalité, toutes les livraisons ne sont pas équivalentes : certaines sont **prioritaires** (livraisons express), tandis que d'autres peuvent être effectuées plus tard dans la journée. L'objectif est donc de concevoir un système de planification capable de trouver un compromis entre la **minimisation du coût global** (distance ou durée totale parcourue) et le **respect des priorités de livraison**.

Ce type de problématique peut être modélisé à l'aide d'un graphe représentant les différents points de livraison et les trajets possibles entre eux. Le cœur du travail consiste à traduire cette situation concrète en un modèle mathématique puis à développer ou adapter des algorithmes capables de proposer une tournée efficace.

Dans ce projet, nous proposons :

- une **modélisation mathématique** du problème sous forme de graphe pondéré, intégrant la distinction entre livraisons normales et express ;
- la **mise en œuvre de plusieurs approches algorithmiques**, allant d'une méthode exacte (brute force) à des heuristiques et métaheuristiques adaptées à des instances plus grandes ;
- l'**évaluation des performances** de ces méthodes selon des indicateurs tels que la distance totale, le respect des délais et la priorisation des clients express.

# Chapitre 2

## Présentation du problème

### 2.1 NP-complétude et NP-difficulté

Un problème est dit dans la classe **P** s'il peut être résolu en temps polynomial, c'est-à-dire que le temps de calcul croît raisonnablement avec la taille des données. Un problème appartient à la classe **NP** (Non-déterministe Polynomial) si, une fois une solution proposée, il est possible de la vérifier en temps polynomial.

Le **problème du voyageur de commerce (TSP)** est l'un des problèmes les plus emblématiques de la complexité combinatoire. Il est **NP-difficile** : le nombre de solutions possibles croît de manière factorielle avec le nombre de sommets.

# Chapitre 3

## Modélisation mathématique

### 3.1 Définition du graphe

Soit :

$$G = (V, A)$$

avec :

$$V = \{0, 1, \dots, n\}, \quad A = \{(i, j) \mid i \neq j\}, \quad n, i, j \in \mathbb{N}$$

- $V$  représente les sommets du graphe, c'est-à-dire les points de livraison et le dépôt (0) ;
- $A$  représente les arêtes reliant deux sommets  $i$  et  $j$ .

On note :

$$T_{ij}, \quad \forall (i, j) \in A$$

le temps de trajet entre deux sommets  $i$  et  $j$ .

### 3.2 Partition des points

Notons la partition :

$$L = V \setminus \{0\}$$

qui représente les points de livraison uniquement.

Ainsi, on peut définir :

- $E$  : ensemble des points de livraisons express ;
- $N$  : ensemble des points de livraisons normales ;

et donc :

$$L = E \cup N$$

### 3.3 Poids des points de livraison

On associe à chaque point  $i \in L$  un poids  $w_i$  :

$$w_i = \begin{cases} w_e & \text{si } i \in E \\ w_n & \text{si } i \in N \end{cases}$$

avec  $w_e > w_n$ , car les points express sont prioritaires.

### 3.4 Coefficients de pondération

On introduit deux coefficients :

- $\alpha$  : pondère la **longueur de la journée** (priorité à la contrainte de durée) ;
- $\beta$  : pondère les **temps d'arrivée** (priorité à la rapidité de livraison).

#### Cas particuliers

- Si l'on souhaite prioriser la **qualité de service** (clients express) :  $\alpha = 1 \Rightarrow$  l'indice de priorité dépend alors de  $\beta$ .
- On peut ajouter une contrainte de durée maximale de tournée, par exemple 8 heures.
- Les valeurs de  $\alpha$  et  $\beta$  peuvent être ajustées selon les priorités du modèle.

### 3.5 Variables de décision

- $x_{ij} \in \{0, 1\}$  : vaut 1 s'il existe une arête entre les sommets  $i$  et  $j$ , 0 sinon ;
- $u_i \in [1, n]$ , pour  $i \in L$ , indique l'ordre de passage du point  $i$  dans la tournée ;
- $t_i$ , pour  $i \in L$ , correspond au temps d'arrivée au point de livraison  $i$ .

### 3.6 Contraintes

#### Continuité du parcours

$$\sum_{j \in V, j \neq i} x_{ij} = 1, \quad \forall i \in L$$
$$\sum_{i \in L, i \neq j} x_{ij} = 1, \quad \forall j \in L$$

Ces deux contraintes garantissent qu'il ne peut exister qu'un seul chemin entre deux points de livraison.

#### Départ et retour au dépôt

$$\sum_{j \in L} x_{0j} = 1 \quad \text{et} \quad \sum_{i \in L} x_{i0} = 1$$

La tournée doit partir du dépôt (0) et y revenir en fin de journée.

### 3.7 Fonction objectif

L'objectif global est de minimiser un indicateur  $I$  représentant le compromis entre la durée totale de la tournée et la rapidité de livraison des clients express :

$$I = \alpha \sum_{(i,j) \in A} T_{ij} x_{ij} + \beta \sum_{i \in L} w_i t_i$$

L'algorithme d'optimisation visera à minimiser  $I$  en trouvant la meilleure combinaison des variables  $x_{ij}$ ,  $u_i$  et  $t_i$ .

### 3.7.1 Relation entre les poids $w_e$ et $w_n$

Afin de pondérer l'importance relative des clients express et normaux dans la fonction objectif, on définit deux coefficients de priorité :  $w_e$  pour les points express et  $w_n$  pour les points normaux.

La relation entre ces deux coefficients dépend de la proportion respective de chaque type de client dans l'instance considérée. On peut fixer les formules suivantes :

$$w_e = \frac{n_{\max} - n_e - 1}{n_{\max} - n_n - 1} + 1 \quad \text{et} \quad w_n = \frac{1}{1 + w_e}$$

où :

- $n_{\max}$  est le nombre total de points de livraison,
- $n_e$  est le nombre de points **express**,
- $n_n$  est le nombre de points **normaux**.

Ces relations garantissent que la condition  $w_e > w_n$  reste toujours vraie, tout en adaptant le différentiel  $\Delta(w_i) = w_e - w_n$  selon la proportion de clients express.

Ainsi :

- si la majorité des points sont express (ex.  $n_e = 90$  sur  $n_{\max} = 100$ ), alors  $w_e$  devient faible, car la priorité relative entre les express perd de son sens ;
- à l'inverse, si peu de points sont express (ex.  $n_e = 10$  sur 100), la différence  $\Delta(w_i)$  devient importante, accentuant la priorité donnée à ces clients.

Ce mécanisme permet d'ajuster dynamiquement la pondération selon la composition du graphe, afin de conserver une hiérarchie de service cohérente entre les deux types de clients.



# Chapitre 4

## Construction et représentation des graphes

### 4.1 Représentation des graphes

La figure ci-dessous illustre la représentation des graphes complets pour différentes tailles :  $n = 5$ ,  $n = 10$  et  $n = 50$ . Chaque sommet correspond à un point du problème :

- le **carré** représente le dépôt ;
- les **triangles** les points de livraison normaux ;
- les **cercles** les points de livraison express.

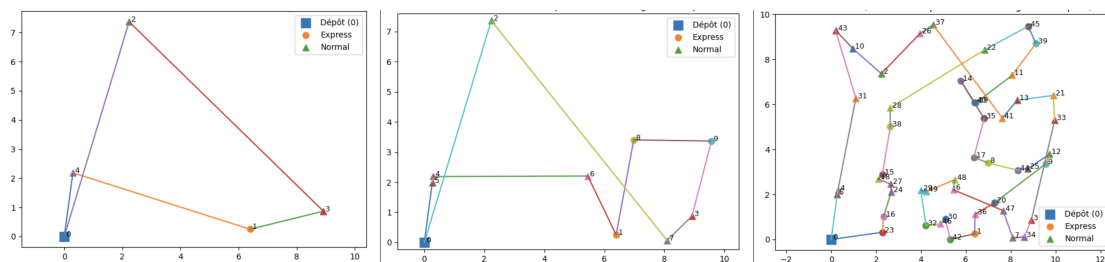


FIGURE 4.1 – Représentation des graphes complets pour différentes tailles ( $n = 5$ ,  $n = 10$ ,  $n = 50$ ).

On observe que chaque graphe est **fermé**, c'est-à-dire que toutes les arêtes relient l'ensemble des sommets entre eux. Chaque arête représente une **distance** entre deux points de livraison. Lorsque la dimension  $n$  augmente, le graphe devient rapidement plus dense, ce qui illustre la croissance de la complexité du problème.

# Chapitre 5

## Résolution du problème

### 5.0.1 Illustration d'une petite instance du problème – Algorithme Brute Force

Pour illustrer la résolution du problème du voyageur de commerce (TSP), nous avons appliqué un **algorithme de recherche exhaustive (brute force)** sur une instance réduite de  $n = 10$  points de livraison (incluant le dépôt). L'algorithme énumère toutes les permutations possibles, en imposant une contrainte : *tous les points « express » doivent être livrés avant les points « normaux »*. Pour chaque permutation, la distance totale parcourue est calculée, et la tournée minimisant cette distance est sélectionnée.

#### Instance générée :

- 1 dépôt (indice 0) ;
- 9 clients dont 3 sont prioritaires (*express*) ;
- Les coordonnées des points ont été générées aléatoirement dans un carré de  $[-50, 50]$ .

#### Résultats obtenus :

- **Tournée optimale (indices)** :  $[0, 4, 6, 2, 9, 7, 8, 1, 5, 3, 0]$
- **Tournée (type des points)** :  $0(D) \rightarrow 4(E) \rightarrow 6(E) \rightarrow 2(E) \rightarrow 9(N) \rightarrow 7(N) \rightarrow 8(N) \rightarrow 1(N) \rightarrow 5(N) \rightarrow 3(N) \rightarrow 0(D)$
- **Distances entre arrêts** :  $[37.7264, 27.0167, 30.4263, 8.7035, 54.8432, 37.8574, 6.8235, 21.0914, 44.0399, 17.1663]$
- **Distance totale minimale** : 285.69 unités de distance

#### Matrice des distances ( $W_{dist}$ ) :

$$W_{dist} = \begin{bmatrix} 0 & 60.89 & 56.11 & 17.17 & 37.73 & 55.97 & 42.38 & 19.16 & 54.21 & 63.11 \\ 60.89 & 0 & 81.59 & 54.23 & 27.21 & 21.09 & 51.16 & 44.68 & 6.82 & 82.46 \\ 56.11 & 81.59 & 0 & 71.23 & 56.64 & 91.00 & 30.43 & 49.83 & 75.92 & 8.70 \\ 17.17 & 54.23 & 71.23 & 0 & 37.66 & 44.04 & 52.12 & 24.94 & 48.24 & 77.48 \\ 37.73 & 27.21 & 56.64 & 37.66 & 0 & 34.63 & 27.02 & 19.14 & 20.62 & 59.04 \\ 55.97 & 21.09 & 91.00 & 44.04 & 34.63 & 0 & 61.64 & 45.21 & 20.73 & 93.66 \\ 42.38 & 51.16 & 30.43 & 52.12 & 27.02 & 61.64 & 0 & 27.24 & 45.55 & 32.05 \\ 19.16 & 44.68 & 49.83 & 24.94 & 19.14 & 45.21 & 27.24 & 0 & 37.86 & 54.84 \\ 54.21 & 6.82 & 75.92 & 48.24 & 20.62 & 20.73 & 45.55 & 37.86 & 0 & 77.23 \\ 63.11 & 82.46 & 8.70 & 77.48 & 59.04 & 93.66 & 32.05 & 54.84 & 77.23 & 0 \end{bmatrix}$$

**Limites et remarques :** L'algorithme brute force permet d'obtenir la solution optimale, mais il présente plusieurs limites :

- Il ne prend pas en compte les contraintes réelles du problème (temps de livraison, durée maximale de tournée, pondération client, fenêtres horaires, etc.) ;

- Sa **complexité est de l'ordre de  $O(n!)$** , car toutes les permutations possibles des points sont testées.

### Analyse de complexité :

$$O(n!) = n \times (n - 1) \times (n - 2) \times \cdots \times 1$$

Cette croissance factorielle rend l'algorithme rapidement inutilisable :

$$10! = 3\,628\,800 \text{ permutations}$$

$$12! = 479\,001\,600 \text{ permutations}$$

Au-delà de  $n \approx 12$ , le temps de calcul devient donc prohibitif, même sur un ordinateur moderne.

**Conclusion :** Cette approche brute force garantit l'**optimalité** de la solution sur de petites instances, mais elle n'est pas exploitable en pratique sur des réseaux réels. Les algorithmes plus avancés (Christofides, insertion gloutonne, heuristiques hybrides, etc.) permettent de trouver des solutions très proches de l'optimum, en un temps de calcul bien plus raisonnable.

## 5.1 Algorithmes d'approximation (Christofides)

### 5.1.1 Algorithme de tournée pondérée avec backbone (Christofides express + normal)

Cet algorithme, noté `tsp_pondere_backbone`, a pour objectif de résoudre le *TSP pondéré* en prenant en compte la priorité des clients **express** tout en respectant une contrainte de durée maximale de tournée. Il combine deux exécutions successives de l'algorithme de **Christofides**, afin de séparer le traitement des livraisons express et normales.

#### Principe général

L'idée principale est de construire une **tournée en deux étapes** :

1. **Phase 1 — Backbone express** : Application de l'algorithme de Christofides uniquement sur les points de livraison express (et le dépôt). On obtient ainsi une première sous-tournée prioritaire dite *backbone*.
2. **Phase 2 — Christofides normal** : Application d'un second Christofides sur l'ensemble des points normaux. Cette sous-tournée vise à compléter le circuit avec les clients non prioritaires.
3. **Phase 3 — Fusion des deux tournées** : Les deux sous-tournées sont **concaténées** en plaçant les points express au début, puis les points normaux. On obtient ainsi un **tour final équilibré**, où les clients express sont toujours livrés en premier.

## Calcul du score global

Une fois la tournée finale établie, on calcule :

$$I = \alpha \cdot T_{\text{total}} + \beta \cdot \sum_i w_i t_i$$

où :

- $T_{\text{total}}$  est la durée totale de la tournée (en heures),
- $w_i = w_{\text{express}}$  si le point est prioritaire, sinon  $w_i = w_{\text{normal}}$ ,
- $\alpha$  et  $\beta$  pondèrent respectivement la durée totale et la priorité de service.

La contrainte de durée maximale est ensuite vérifiée :

$$T_{\text{total}} \leq T_{\text{max}} = 8 \text{ heures}$$

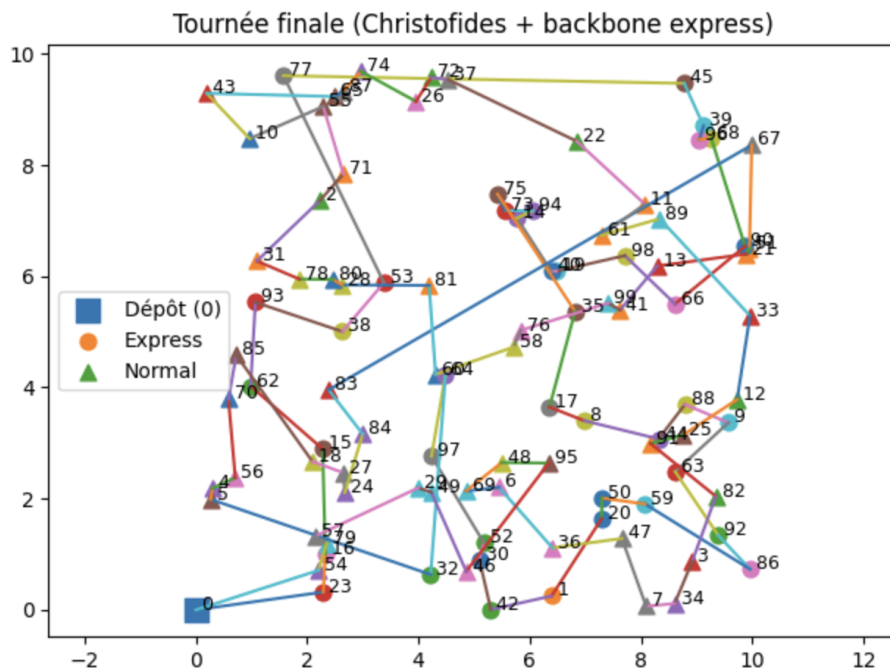
Si cette limite est dépassée, l'algorithme **supprime progressivement les derniers clients** jusqu'à ce que le retour au dépôt soit garanti avant la fin de la journée.

## Paramètres utilisés

- $n = 100$  : nombre total de points de livraison.
- $r_{\text{express}} = 0.4$  : 40% des clients sont express.
- $\alpha = 1, \beta = 1$  : équilibre entre durée et priorité.
- $v = 30 \text{ km/h}$  : vitesse moyenne de déplacement.
- $T_{\text{max}} = 8 \text{ h}$  : durée maximale de la tournée.
- $w_{\text{express}} = \frac{(n-n_e-1)}{(n-n_n-1)} + 1, \quad w_{\text{normal}} = \frac{1}{1+w_{\text{express}}}.$

## Résultats obtenus (exemple sur 100 points)

- **Durée totale** :  $T_{\text{total}} = 4.107 \text{ h}$
- **Score final** :  $I = 147.959 = \alpha \times 4.107 + \beta \times 143.852$
- **Contrainte respectée** :  $T_{\text{total}} \leq 8 \text{ h}$
- **Observation** : Les clients express sont systématiquement servis avant les normaux, assurant une meilleure qualité de service sans dépasser la durée maximale.



## Analyse

Cet algorithme présente un bon compromis entre :

- la **qualité de service** (grâce à la priorisation des express),
- et l'**efficacité logistique** (en limitant la durée de la tournée).

Sa complexité est dominée par celle de Christofides, soit environ :

$$O(n^3)$$

ce qui le rend exploitable pour des instances de plusieurs centaines de points, contrairement à l'approche *brute force* en  $O(n!)$ . Cette méthode est donc **efficace, réaliste et adaptable**, en particulier dans un contexte de logistique urbaine avec contraintes horaires.

### 5.1.2 Algorithme hybride : backbone de Christofides et insertion gloutonne

Afin de dépasser les limites de la recherche exhaustive, un algorithme plus performant a été développé. Celui-ci combine deux principes complémentaires :

1. la construction d'un **backbone optimal** sur les clients prioritaires (*express*) grâce à l'algorithme de **Christofides**, garantissant une solution proche de l'optimum pour ce sous-graphe ;
2. l'**insertion gloutonne des clients normaux**, réalisée selon la fonction objectif  $I$ , qui intègre la durée totale et la priorité des clients.

**Fonction objectif.** L'objectif global est défini par :

$$I = \alpha \cdot T_{\text{total}} + \beta \sum_{i \in L} w_i \cdot t_i$$

où :

- $T_{\text{total}}$  est la durée totale de la tournée ;
- $w_i$  est le poids du client  $i$  (3 pour les express, 1 pour les normaux) ;
- $t_i$  est l'heure d'arrivée au client  $i$  ;
- $\alpha$  et  $\beta$  permettent de pondérer respectivement la durée et la priorité des livraisons.

**Étapes de l'algorithme :**

1. **Construction du sous-graphe express** : application de l'algorithme de Christofides sur le dépôt et les clients express pour obtenir une tournée optimisée (*backbone*).
2. **Insertion gloutonne des clients normaux** : chaque client normal est inséré à la position minimisant la variation  $\Delta I$ , tout en respectant la contrainte de durée maximale de tournée (8 h).
3. **Évaluation finale** : calcul du coût  $I$  global et de ses composantes  $\alpha \cdot T_{\text{total}}$  et  $\beta \sum w_i t_i$ .

## Paramètres utilisés pour l'expérience :

|                                      |                                           |
|--------------------------------------|-------------------------------------------|
| $\alpha = 1.0$                       | (influence sur la durée globale)          |
| $\beta = 1.0$                        | (influence sur la priorité des clients)   |
| $w_{\text{express}} = 3.0$           | (importance des clients prioritaires)     |
| $w_{\text{normal}} = 1.0$            | (importance des clients standards)        |
| $\text{max\_hours} = 8.0 \text{ h}$  | (durée maximale de tournée)               |
| $\text{speed} = 30 \text{ unités/h}$ | (conversion distance $\rightarrow$ temps) |
| $n = 100$                            | (nombre total de points de livraison)     |
| $\text{ratio\_express} = 0.4$        | (40 % de clients express)                 |

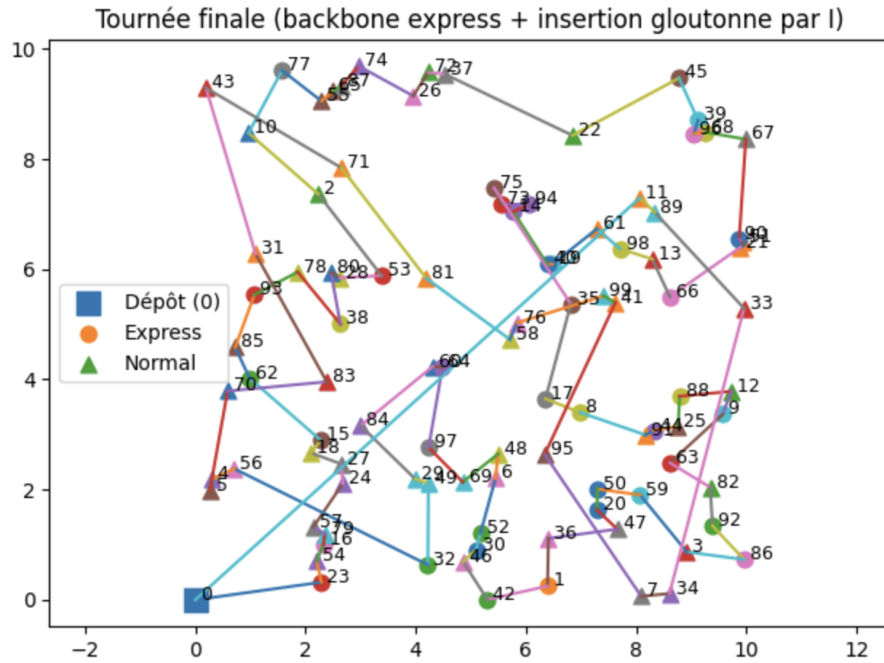
**Résultat obtenu (exemple pour  $n = 100$ ).** L'algorithme a produit la tournée suivante :

Tour (indices) =  $[0, 23, 54, 16, 79, 57, 24, 27, 18, 15, \dots, 11, 0]$

avec une **durée totale de tournée**  $T_{\text{total}} = 3.913 \text{ h}$  et un coût global :

$$I_{\text{final}} = 249.46 = \underbrace{3.91}_{\alpha T} + \underbrace{245.54}_{\beta \sum w_i t_i}$$

La figure suivante illustre la tournée finale obtenue :



Complexité. L'algorithme possède une complexité bien plus faible que la recherche exhaustive :

$$\text{Complexité globale} \approx O(m^3) + O((n - m)^2)$$

où :

—  $m$  = nombre de clients express (backbone de Christofides),

- $n - m$  = nombre de clients normaux insérés gloutonnement.

Le premier terme ( $O(m^3)$ ) correspond à l'algorithme de Christofides (fondé sur le couplage parfait minimal), tandis que le second ( $O((n - m)^2)$ ) vient des insertions successives.

Ainsi, contrairement au brute force ( $O(n!)$ ), cet algorithme reste exploitable pour des instances de taille  $n = 100$  voire plus, tout en respectant les contraintes réelles du problème.

**Conclusion.** L'algorithme hybride backbone-insertion constitue un compromis efficace entre **qualité de solution** et **temps de calcul**. Il permet d'intégrer à la fois les contraintes de durée, de priorité client et d'ordre de service, tout en restant applicable à des instances réalistes de grande taille. Il sert ainsi de base à une approche plus complète de planification des tournées optimisées.

## 5.2 Comparaison des complexités algorithmiques

Pour visualiser la croissance des temps de calcul selon le nombre de sommets, nous avons comparé cinq algorithmes : Prim, Kruskal, Brute Force, Christofides et notre approche **Hybride**. Le graphique suivant illustre leur évolution asymptotique :

**Analyse des résultats :**

- **Prim** ( $O(n^2)$ ) et **Kruskal** ( $O(n^2 \log n)$ ) affichent une croissance polynomiale faible : ils sont efficaces mais ne résolvent pas directement le TSP.
- **Brute Force** ( $O(n!)$ ) explose très rapidement : la courbe devient exponentielle dès  $n > 10$ , confirmant son inutilisabilité pour des grandes instances.
- **Christofides** ( $O(n^3)$ ) offre un bon compromis entre précision et performance, garantissant une solution proche de l'optimum.
- L'**algorithme Hybride** ( $O(n^3 + n^2)$ ) conserve la structure polynomiale de Christofides tout en intégrant les contraintes réelles (priorité, durée maximale). Sa croissance reste maîtrisée même pour  $n = 100$  points.

**Conclusion :** L'analyse graphique met en évidence la différence majeure entre les approches exactes et heuristiques. Alors que le **Brute Force** devient rapidement inapplicable, les méthodes **Christofides** et surtout l'**Hybride** permettent de traiter efficacement des instances de grande taille tout en conservant une qualité de solution élevée. Notre algorithme final offre donc un excellent compromis entre *temps de calcul* et *respect des contraintes opérationnelles*.

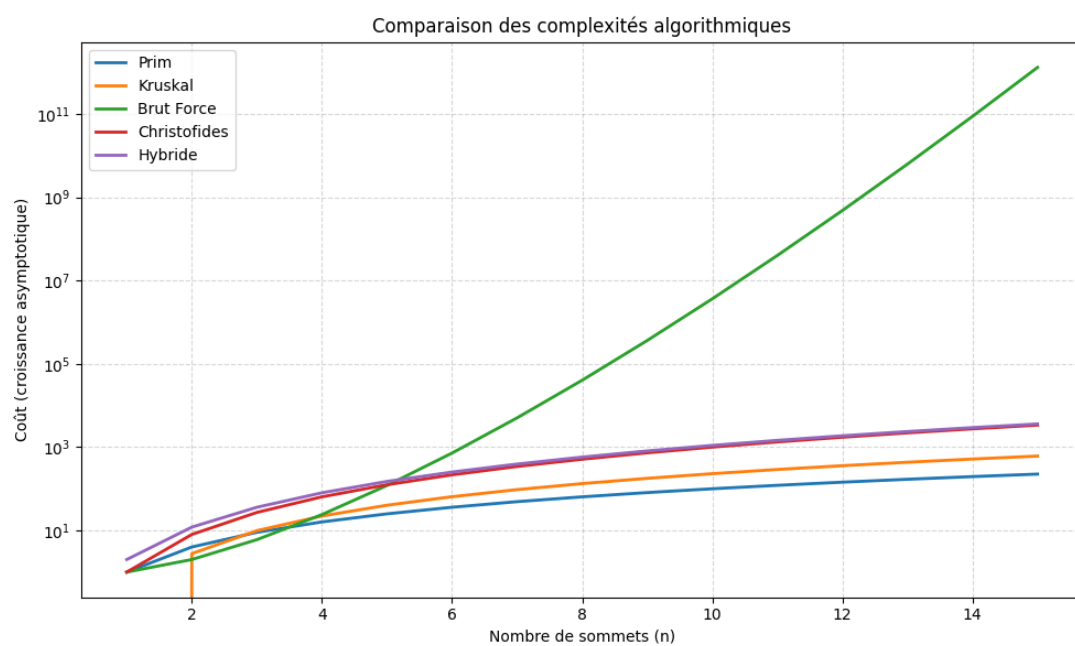


FIGURE 5.1 – Comparaison des complexités algorithmiques des différents algorithmes TSP.



# Chapitre 6

## Métaheuristique : principe général et application

### 6.1 Principe général

La métaheuristique des **colonies de fourmis** (**Ant Colony Optimization, ACO**) s'inspire du comportement collectif observé chez les fourmis dans la nature. Lorsqu'elles recherchent une source de nourriture, les fourmis déposent sur leur chemin une *phéromone* qui attire leurs congénères. Les trajets efficaces concentrent progressivement davantage de phéromones, devenant ainsi les plus empruntés par la colonie.

Cette approche repose sur trois mécanismes fondamentaux :

- **Exploration** : chaque fourmi construit un trajet en choisissant sa prochaine étape selon une probabilité qui combine la visibilité (heuristique locale) et la quantité de phéromone  $\tau_{ij}$  ;
- **Exploitation** : les chemins performants accumulent davantage de phéromones, renforçant leur attractivité pour les itérations suivantes ;
- **Évaporation** : une partie des phéromones s'estompe à chaque itération, évitant la fixation sur un minimum local et maintenant l'exploration.

### 6.2 Application au TSP pondéré (cas du livreur)

Dans notre modèle de tournée pondérée pour un livreur, chaque fourmi correspond à un livreur virtuel qui construit une tournée complète parmi les points de livraison (le dépôt est l'indice 0). La probabilité que la fourmi en  $i$  choisisse  $j$  comme prochaine étape est donnée par :

$$P_{ij} = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum_{k \in \text{non visités}} (\tau_{ik})^\alpha \cdot (\eta_{ik})^\beta},$$

où :

- $\tau_{ij}$  est l'intensité de la phéromone sur l'arête  $(i, j)$  ;
- $\eta_{ij} = 1/d_{ij}$  est la visibilité (inverse de la distance) ;
- $\alpha$  et  $\beta$  pondèrent respectivement l'influence des phéromones et de la visibilité.

Pour tenir compte de la priorité des clients (express vs normal) nous adaptons la visibilité en introduisant un facteur de pondération lié au type de client :

$$\eta_{ij} = \frac{1}{d_{ij} \cdot (1 + \lambda \cdot w_i)},$$

où  $w_i$  est le poids du client  $i$  (plus élevé pour les express) et  $\lambda$  un coefficient d'intensité. De cette manière, un client express (grand  $w_i$ ) réduit localement la visibilité d'arêtes très longues menant à des non-prioritaires, favorisant la construction de tournées où les express sont servis tôt.

L'objectif minimisé par l'ACO dans notre implémentation est exactement le même que pour les autres algorithmes du projet :

$$I = \alpha \cdot T_{\text{total}} + \beta \cdot \sum_i w_i t_i,$$

où  $T_{\text{total}}$  est le temps total de la tournée (aller + retour au dépôt), et  $t_i$  le temps d'arrivée du livreur au client  $i$ .

## 6.3 Implémentation résumée

Notre implémentation ACO (fichier `aco.py`) suit le schéma classique :

1. initialisation des phéromones  $\tau_{ij} = \tau_0$  et calcul de la matrice heuristique  $\eta_{ij}$  ;
2. pour chaque itération :
  - chaque fourmi construit une tournée à partir du dépôt en tirant au sort la prochaine arête selon la distribution  $P_{ij}$  adaptée au poids des clients ;
  - on évalue la tournée avec l'objectif  $I$  (y compris pénalité douce ou contrainte dure sur  $T_{\text{total}}$ ) ;
  - évaporation des phéromones (multiplication par  $(1 - \rho)$ ) ;
  - dépôt de phéromone proportionnel à la qualité (inverse de  $I$ ) sur les arêtes de la meilleure solution globale.
3. sortie : meilleure tournée trouvée, coût  $I$ , décomposition en termes  $\alpha$  et  $\beta$ , temps total et distances.

Les principaux paramètres expérimentaux sont :

$\alpha, \beta, \rho$  (évaporation),  $Q$  (quantité déposée), nombre de fourmis  $m$ , itérations  $k$ .

## 6.4 Résultats expérimentaux (exemple)

L'algorithme ACO a été exécuté sur une instance de 100 points (dont 40 express) avec les paramètres suivants :  $\alpha = 1.0$ ,  $\beta = 3.0$ ,  $\rho = 0.5$ ,  $Q = 1.0$ , vitesse  $v = 30$  unités/h,  $k = 200$  itérations et  $m = 100$  fourmis.

Le meilleur résultat obtenu (exécution d'exemple) est :

Objectif  $I = 299.329$

Temps total  $T = 4.993$  h ; distance = 149.801 unités

Ordre de visite (indices):

```
0 -> 97 -> 29 -> 49 -> 69 -> 6 -> 46 -> 32 -> 23 -> 54 -> 79 -> 24 -> 18
-> 15 -> 27 -> 16 -> 30 -> 42 -> 7 -> 86 -> 92 -> 3 -> 34 -> 47 -> 8
-> 50 -> 20 -> 59 -> 12 -> 9 -> 82 -> 63 -> 91 -> 44 -> 25 -> 95 -> 17
-> 64 -> 60 -> 81 -> 53 -> 19 -> 40 -> 89 -> 11 -> 13 -> 94 -> 61 -> 98
-> 99 -> 41 -> 68 -> 96 -> 39 -> 45 -> 58 -> 76 -> 1 -> 36 -> 88 -> 33
-> 51 -> 90 -> 21 -> 67 -> 66 -> 35 -> 14 -> 73 -> 75 -> 22 -> 37 -> 26
```

-> 77 -> 65 -> 87 -> 55 -> 74 -> 71 -> 2 -> 31 -> 38 -> 93 -> 85 -> 62  
-> 5 -> 56 -> 4 -> 70 -> 83 -> 84 -> 57 -> 78 -> 80 -> 28 -> 10 -> 43  
-> 72 -> 48 -> 52 -> 0

Visités: `express` = 40, `normal` = 59

Remarques :

- la durée totale est inférieure à la contrainte de 8 h (4.993 h) ; l’algorithme a donc trouvé une tournée faisable ;
- la décomposition de l’objectif montre que la part liée à la priorité ( $\beta$ -term) domine généralement dans nos instances si  $\beta$  est élevé ; cela correspond au souhait de servir tôt les clients express.

## 6.5 Complexité et intérêt pratique

La complexité opérationnelle par itération est en pratique de l’ordre de :

$$O(m \cdot n^2)$$

(chaque fourmi construit une tournée coûteuse en opérations sur la matrice des distances, et la mise à jour des phéromones est proportionnelle au nombre d’arêtes visitées). Sur  $k$  itérations, on obtient donc :

$$O(k \cdot m \cdot n^2).$$

Avec un choix raisonnable de  $k$  et  $m$  (par exemple  $k \in [100, 1000]$ ,  $m \approx n$  ou  $m < n$ ), l’ACO reste applicable à des instances bien plus grandes que celles accessibles par la recherche exhaustive et souvent compétitive en qualité avec Christofides et l’algorithme hybride.

## 6.6 Conclusion

La métaheuristique ACO constitue une alternative robuste et flexible pour notre problème de tournée pondérée avec priorités. Elle est particulièrement pertinente lorsque l’on souhaite :

- traiter des instances de grande taille ou dynamiques,
- intégrer des préférences (pondérations express) directement dans la phase de construction,
- obtenir plusieurs bonnes solutions stochastiques exploitables pour l’analyse comparative.

Pour notre projet, l’Algorithme hybride : backbone de Christofides et insertion gloutonne reste une solution de départ rapide et stable, mais l’ACO apporte une capacité d’optimisation progressive et d’adaptation qui s’avère précieuse sur de grands jeux de données.

# Chapitre 7

## Indicateur de performance

Afin de comparer les différentes approches de manière quantitative, deux indicateurs ont été définis : l'**Express Priority Index (EPI)** et le **Duration Usage Index (DU)**. Ces indicateurs permettent respectivement d'évaluer la priorité accordée aux clients express et l'efficacité d'utilisation de la durée maximale de tournée.

### 7.0.1 Express Priority Index (EPI)

L'*Express Priority Index* est défini par la formule suivante :

$$\text{EPI} = \frac{\sum_{i \in E} w_i \left(1 - \frac{\text{rang}(i)}{n-1}\right)}{\sum_{i \in E} w_i}$$

Décomposons cette formule :

- Le terme  $1 - \frac{\text{rang}(i)}{n-1}$  est proche de 1 si le point express  $i$  est livré tôt dans la tournée. Il agit donc comme un *coût de priorité* : plus cette valeur est élevée, plus le client express a été servi rapidement.
- La multiplication de ce coût par  $w_i$  (le poids des clients express) puis la division par la somme des  $w_i$  revient à effectuer une **moyenne pondérée** des priorités.
- Comme chaque terme est compris entre 0 et 1, la valeur finale de l'EPI est également bornée dans  $[0, 1]$ .

Ainsi, un EPI proche de 1 indique que les points express ont été livrés très tôt dans la tournée. Cet indicateur permet donc de quantifier le niveau de priorité réellement accordé aux clients express par un algorithme donné.

### 7.0.2 Duration Usage Index (DU)

Le *Duration Usage Index* est défini par :

$$\text{DU} = \frac{\sum_{(i,j) \in A} T_{ij}}{T_{\max}}$$

Cet indice mesure la **proportion de la durée finale de la tournée** par rapport à la durée maximale autorisée  $T_{\max}$ . Plus la valeur de  $\text{DU}$  se rapproche de 1, plus la journée du livreur a été exploitée efficacement, c'est-à-dire que la tournée utilise presque toute la plage horaire disponible sans la dépasser.

### 7.0.3 Analyse comparative

Parmi l'ensemble des algorithmes développés, la comparaison portera principalement sur :

- **Algorithme 1 : Backbone express + insertion gloutonne**
- **Algorithme 2 : Concaténation express + normaux**

On peut dégager les tendances suivantes :

- L'EPI de l'algorithme 2 est toujours supérieur à celui de l'algorithme 1, car il correspond au *cas idéal* où tous les express sont livrés avant les normaux.
- Cependant, lorsque  $n$  augmente et que le ratio de clients express reste fixe, les deux EPI deviennent relativement proches.
- En revanche, le  $DU$  de l'algorithme 2 reste systématiquement plus élevé que celui de l'algorithme 1.

**Conclusion :** Si  $EPI_2 > EPI_1$  mais que l'écart entre les deux reste faible lorsque  $n$  augmente, et que  $DU_2 > DU_1$ , alors l'algorithme 1 est plus complet et optimal. En effet, il parvient à **prioriser les clients express tout en optimisant la durée totale de tournée**, ce que l'algorithme 2 ne peut pas réaliser. Ainsi, la **limite de clients livrables** par l'algorithme 1 est strictement supérieure à celle de l'algorithme 2.

# Chapitre 8

## Analyse critique et conclusion

### 8.0.1 Limites de la simulation

- Notre simulation **ne prend pas en compte les temps d'arrêt** du livreur (chargement, déchargement, pauses, etc.), ce qui **fausse légèrement la durée réelle** de la tournée.
- De même, nous n'avons pas intégré **les conditions de trafic**, qui influencent fortement les temps de trajet.
- Ces aspects ont été jugés trop complexes à modéliser dans le cadre du projet, mais pourraient être **ajoutés dans une version future** pour se rapprocher d'une situation réelle.

### 8.0.2 Optimisation des paramètres $\alpha$ et $\beta$

- Notre algorithme pourrait encore être amélioré grâce à une **recherche automatique du meilleur couple de pondération**  $(\alpha, \beta)$ .
- Lorsqu'il reste du temps disponible avant d'atteindre la durée maximale (8 h), il serait possible **d'ajuster dynamiquement ces coefficients** pour réduire l'indice global  $I$  :
  - **Augmenter**  $\beta \Rightarrow$  prioriser les points express.
  - **Diminuer**  $\alpha \Rightarrow$  réduire le poids de la durée totale.
- Cette approche permettrait d'exploiter au mieux le temps restant pour **améliorer la qualité de service** sans dépasser la contrainte horaire.

### 8.0.3 Présentation des trois algorithmes développés

#### Algorithme 1 : Brute force

- Teste **toutes les combinaisons possibles** de trajets pour trouver la solution optimale.
- **Efficace uniquement** pour un petit nombre de livraisons ( $n \leq 11$ ).
- Complexité **factorielle** ( $O(n!)$ ), le rendant vite inutilisable pour de grands ensembles.
- Sert principalement de **référence de performance** pour comparer les autres méthodes.

### Algorithme 2 : Concaténation de backbone express + backbone normal

- Exécute un **Christofides séparé** pour les points express, puis un autre pour les points normaux.
- Les deux tournées sont ensuite **concaténées** (express  $\rightarrow$  normal).
- **Avantages :**
  - Les clients express sont servis **en premier**, réduisant la somme pondérée des temps d'arrivée (partie droite de  $I$ ).
- **Inconvénients :**
  - **Durée totale plus élevée**, car la priorité est donnée au service client.
  - $\alpha$  et  $\beta$  **n'ont aucun effet**, puisque l'ordre express  $\rightarrow$  normal est fixe.
- Cet algorithme met donc **la qualité de service au-dessus de l'efficacité globale**.

### Algorithme 3 : Backbone express + insertion gloutonne

- Repose sur un **Christofides appliqué aux points express**, formant une colonne vertébrale (*backbone*).
- Les **points normaux** sont ensuite **insérés de manière gloutonne** pour compléter la tournée.
- Il s'agit d'une solution **polyvalente** :
  - Indice  $I$  relativement faible.
  - Durée de tournée optimisée.
- C'est le meilleur compromis entre :
  - **Efficacité logistique** (minimiser la durée totale),
  - **Qualité de service** (livrer les express en priorité).
- Avec  $\alpha = \beta = 1$ , il donne des résultats équilibrés.
- Une **fonction d'ajustement de**  $(\alpha, \beta)$  pourrait renforcer la priorité des express si du temps reste disponible.

### Algorithme 4 : Colonies de fourmis (ACO)

- **Principe** : des fourmis artificielles construisent des tournées en favorisant les arcs à forte *phéromone*  $\tau_{ij}$  et/ou à forte *visibilité*  $\eta_{ij}$ . À chaque itération, on met à jour les phéromones selon la qualité des solutions trouvées.
- **Complexité** :  $\mathcal{O}(k m n^2)$  (construction des  $m$  tournées + mises à jour), en pratique quasi-linéaire en  $k$  et  $m$ .
- **Avantages :**
  - Bon compromis qualité/temps sur des tailles  $n$  moyennes à grandes.
  - Très adaptable à différents objectifs (ici  $I$ ) et contraintes (priorité express).
  - Parallélisable (les fourmis construisent indépendamment).
- **Inconvénients :**
  - Sensible au réglage de  $a_{\text{phero}}, b_{\text{visu}}, \rho, Q$ .
  - Risque de stagnation (exploration insuffisante) sans évaporation suffisante ou diversification.
  - Pas de garantie d'optimalité.

| Algorithme                               | Principe                                     | Temps de calcul           | Durée totale     | Indice $I$                      | Priorisation express              | Commentaire                                                    |
|------------------------------------------|----------------------------------------------|---------------------------|------------------|---------------------------------|-----------------------------------|----------------------------------------------------------------|
| <b>1. Backbone + Insertion gloutonne</b> | Christofides express + insertion des normaux | Moyenne                   | Faible à moyenne | Bon compromis                   | Oui, modulable via $\alpha/\beta$ | Solution la plus équilibrée                                    |
| <b>2. Concaténation express + normal</b> | Deux Christofides séparés                    | Faible                    | Élevée           | Faible (sur la somme de droite) | Totale                            | Favorise la qualité de service au détriment de la durée        |
| <b>3. Brute force</b>                    | Recherche exhaustive                         | Très long                 | Optimale         | Optimale                        | Oui (selon configuration)         | Référence théorique pour petits jeux de données                |
| <b>4. ACO</b>                            | Métaheuristique par colonies de fourmis      | Moyen à élevé (selon k,m) | Moyenne          | Bon compromis                   | Oui (via poids)                   | Apprentissage adaptatif, scalable, efficace sur grands graphes |

TABLE 8.1 – Comparaison synthétique des trois algorithmes développés.

## 8.0.4 Comparaison synthétique

## 8.0.5 Conclusion

Notre approche permet d'obtenir des **résultats cohérents et équilibrés** pour le problème du TSP pondéré. L'**algorithme 1** apparaît comme le plus robuste, combinant :

- Rapidité d'exécution,
- Flexibilité (grâce aux coefficients  $\alpha$  et  $\beta$ ),
- Respect de la contrainte de durée maximale.

L'ajout d'une **fonction de recherche dynamique du couple**  $(\alpha, \beta)$  permettrait d'améliorer encore la **qualité de service** sans dépasser la durée maximale. Cette extension rendrait la simulation **plus réaliste, plus performante et plus proche des besoins opérationnels** d'une tournée réelle.



# Annexe — Algorithmes et sources

## **TSP : algorithme de Christofides (métrique)**

[https://fr.wikipedia.org/wiki/Algorithme\\_de\\_Christofides](https://fr.wikipedia.org/wiki/Algorithme_de_Christofides)

## **TSP : brute force**

<https://www.routific.com/blog/travelling-salesman-problem>

## **Méta-heuristique (ACO)**

<https://learn.microsoft.com/fr-fr/archive/msdn-magazine/2012/february/test-run-ant-colony-optimization>